

PBL 04: AI LLM 진단 절차와 대응 방안

정하은

1. 개요

인공지능 기술의 발전과 함께 대규모 언어 모델(LLM, Large Language Model)은 현대 사회의 핵심 기술로 자리 잡았다. LLM은 자연어 처리 능력을 바탕으로 다양한 분야에 혁신을 가져왔지만, 동시에 기존의 IT 시스템과는 다른 고유한 보안 취약점들을 노출시키고 있다.

본 보고서는 LLM의 작동 원리를 간략히 설명하고, OWASP Top 10 for LLM Applications 2025을 기반으로 주요 보안 취약점을 분석한다. 또한, 각 취약점에 대한 대응 방안을 제시하여 LLM을 안전하게 활용하기 위한 방안을 모색하고자 한다.

2. AI LLM의 이해

LLM(Large Language Model)은 방대한 양의 텍스트 데이터를 학습하여 인간의 언어를 이해하고 생성하는 AI 모델이다. ChatGPT, Gemini 등이 대표적인 예시이며, 이들은 트랜스포머(Transformer)라는 신경망 아키텍처를 기반으로 다음 단어를 예측하는 방식으로 문장을 완성해 나간다.

이러한 LLM은 챗봇, 콘텐츠 생성, 코드 작성 등 다양한 분야에서 활용되며 높은 유용성을 입증할 수 있었다. 그러나 LLM은 데이터와 상호작용하는 방식 때문에 기존의 애플리케이션과는 다른 취약점을 가질 수밖에 없다. 전통적인 웹/모바일 앱과 달리 LLM은 프롬프트와 데이터가 주요 공격 표면이 되며, 비정형 입력과 외부 플러그인 기능 등으로 인해 예상치 못한 보안 위험이 발생할 수 있다.

3. OWASP LLM Top 10 취약점 정리

OWASP는 LLM 기반 애플리케이션의 주요 보안 취약점을 정리하여 Top 10을 발표했다. 이는 다음과 같은 취약점들로 구성되어 있다.

코드	취약점	설명	영향
LLM01	프롬프트 주입	악성 입력으로 모델 동작 변경	민감정보 노출, 권한 우회
LLM02	민감 정보 유출	출력에 개인정보/비밀 포함	데이터 유출, 법적 문제
LLM03	데이터 중독	학습·파인튜닝 데이터 변조	모델 오작동, 악성 출력
LLM04	악성 플러그인 호출	LLM이 악성 API 호출	시스템 침해
LLM05	취약한 플러그인 보안	인증·권한 제어 미흡	무단 접근
LLM06	과도한 위임	LLM이 위험한 작업 수행	비인가 동작
LLM07	시스템 프롬프트 유출	내부 지침·민감데이터 노출	보안 우회, 공격 정보 제공

LLM08	벡터·임베딩 취약점	벡터DB/RAG 데이터 유출	정보 노출, 저작권 침해
LLM09	허위 정보	사실처럼 보이는 오류 생성	잘못된 의사결정, 평판 손상
LLM10	무제한 소비	과도한 요청으로 자원 고갈	서비스 장애(DoS)

4. LLM 보안 진단 및 대응 방안

앞서 살펴본 OWASP LLM Top 10 취약점들은 LLM 애플리케이션을 안전하게 운영하기 위해 반드시 고려해야 할 항목이다. 다음은 OWASP LLM Top 10과 SK 쉐더스에서 제공하는 취약점들을 사전에 진단하고, 효과적으로 방어하기 위한 절차와 대응 방안이다.

4.1. AI LLM 보안 진단 절차

1. 사전 준비 (Pre-Engagement)

- **구조 및 흐름 파악** : LLM 애플리케이션의 전체 구조도, 데이터 흐름도, 운영 정책 문서를 수집하여 시스템의 작동 방식을 이해한다.
- **연계 요소 식별** : LLM과 상호작용하는 모든 외부 연계 요소(플러그인, API, DB, RAG용 벡터DB 등)의 목록을 작성한다.
- **테스트 데이터 준비** : 민감정보가 마스킹 처리된 샘플 데이터셋을 준비하여 테스트에 활용한다.
- **진단 목적 및 범위 설정** : 기능별, 보안 영역별로 테스트 범위를 명확하게 정의한다.

2. 자산 식별 (Asset Identification)

- **모델 식별** : 사용 중인 LLM의 이름, 버전, 배포 형태(API, 온프레미스 등)를 명확히 파악한다.
- **데이터 흐름 파악** : LLM의 입력 데이터 출처(사용자, 외부 API, DB)와 출력 데이터의 전송 경로를 분석한다.
- **환경 설정 점검** : 인증 방식, 접근 제어 정책, 로깅 및 모니터링 체계의 유무를 점검한다.

3. 진단 범위 정의 (Scope Definition)

- 모델 입력/출력 제어
- 시스템 프롬프트 보호 여부
- 외부 API 호출 제어
- 데이터 무결성 검증
- 리소스 제한 설정
- 법규 및 규제 준수 여부

4. 위협 모델링 (Threat Modeling)

- **다양한 LLM 취약점 진단 가이드 기반 위협 매핑** : 식별된 자산과 데이터 흐름에 OWASP LLM Top 10의 취약점 등을 매핑하여 잠재적인 위협을 도출한다.
- **공격 표면 식별** : 사용자 입력 프롬프트, 플러그인/DB API 호출, RAG(VectorDB) 문서 검색

결과 등 공격자가 접근할 수 있는 모든 지점을 식별한다.

- 공격 경로 도출 : STRIDE, DREAD, PASTA와 같은 보안 분석 기법을 활용해 공격자가 시스템을 침해할 수 있는 구체적인 경로를 도출한다.

5. 테스트 계획 수립 (Test Planning)

- 시나리오 설계 : 악성 프롬프트 삽입 테스트, 벡터DB 민감 정보 질의 테스트, 플러그인 API 무단 호출 테스트 등 구체적인 공격 시나리오를 설계한다.
- 테스트 환경 구축 : 운영 환경과 동일하되, 민감 데이터는 제거하거나 마스킹 처리된 테스트 환경을 구축한다.
- 성공/실패 기준 정의 : "LLM이 내부 API 키를 출력하면 실패"와 같이 테스트의 성공/실패 기준을 명확하게 정의한다.

6. 보안 점검 수행 (Security Assessment)

- 입력 검증 점검 : XSS, SQL Injection, Prompt Injection 시나리오를 적용하여 입력 검증 로직을 테스트한다.
- 출력 검증 점검 : LLM의 출력에 민감 정보, 악성 코드, 유해한 링크가 포함되는지 검사한다.
- 플러그인/API 보안 점검 : 플러그인/API 호출에 대한 인증, 권한 제어, 호출 제한 여부를 점검한다.
- 데이터 보안 점검 : 학습 데이터의 암호화 상태, 접근 제어, 백업 정책 등을 점검한다.

7. 결과 분석 및 보고 (Analysis & Reporting)

- 취약점 목록화 : 발견된 취약점의 심각도, 영향, 재현 절차를 포함하여 목록화한다.
- 취약점 심각도 분류 : Critical, High, Medium, Low 등 심각도에 따라 취약점을 분류한다.
- 개선 권고안 제시 : 취약점을 해결하기 위한 구체적인 권고안과 우선순위를 제시한다.
- 맞춤 보고서 작성 : 경영진, 개발팀, 보안팀 등 각 대상에 맞는 맞춤형 보고서를 작성한다.

4.2. AI LLM 보안 대응 방안

1. 입력 검증 (Input Validation)

- 악성 프롬프트 탐지 필터를 적용하여 정규식이나 키워드를 기반으로 악성 입력을 차단한다.
- HTML/스크립트 태그를 제거하여 XSS 공격을 방지한다.
- 입력 길이를 제한하고, 비정상적인 패턴의 입력을 거부한다.
- 프롬프트 체인 내부에 있는 시스템 지침(System Prompt)이 외부에 노출되지 않도록 주의한다.

2. 출력 검증 (Output Validation)

- 민감 정보(전화번호, 주민등록번호, API Key 등)를 자동으로 마스킹하는 후처리 모듈을 구축한다.
- LLM이 생성한 HTML/JavaScript 코드는 제거하거나, 샌드박스(Sandbox) 환경에서만 실행되도록 한다.
- 사용자에게 출력하기 전에 보안 검증 모듈을 거쳐 악성 링크나 파일 확장자 등을 필터링한다.

3. 권한 최소화 (Principle of Least Privilege)

- LLM이 외부 플러그인/API를 호출할 때 필요한 **최소한의 권한**만 부여한다.
- 호출 가능한 API 목록을 **화이트리스트**로 관리하여 의도치 않은 호출을 차단한다.
- 데이터베이스 질의 범위를 제한하고(SELECT * 사용 금지), 벡터DB에서 민감 데이터 문서의 검색을 차단한다.

4. 데이터 보호 (Data Protection)

- 학습 및 파인튜닝 데이터를 **암호화**하고, SHA-256 해시 등을 통해 데이터의 무결성을 검증한다.
- 데이터 전송 구간에는 반드시 **TLS/SSL 암호화**를 적용한다.
- 민감 정보는 RAG(Retrieval-Augmented Generation) 데이터베이스에 저장하지 않거나, 저장 전에 익명화한다.

5. 리소스 관리 (Resource Management)

- LLM API에 **요청 빈도 제한(Rate Limiting)**을 설정하여 DoS 공격을 방지한다.
- 응답 토큰 수를 제한하여 LLM의 비정상적인 자원 소모를 막는다.
- **WAF(Web Application Firewall)**와 연계하여 비정상적인 대량 요청을 자동으로 차단한다.

6. 모니터링 및 로깅 (Monitoring & Logging)

- 모든 입력/출력, 플러그인 호출, API 요청을 상세하게 **로깅**한다.
- IDS/IPS와 연계하여 LLM의 행동 패턴에서 **실시간 이상 징후**를 탐지한다.
- 규제 준수 목적을 위해 감사 로그를 안전하게 보존한다.

7. 보안 정책 및 교육 (Policy & Awareness)

- 개발자 및 운영자에게 **LLM 보안 교육**을 정기적으로 실시한다.
- 안전한 프롬프트 설계 가이드라인을 제정하고, 보안 점검 체크리스트를 유지 및 업데이트한다.
- 신규 플러그인/데이터셋 도입 전 사전 보안 검토를 의무화한다.
- 정기적인 모의 침투 테스트를 수행하여 취약점을 선제적으로 발견하고 개선한다.

5. 결론

LLM 기술은 개발자에게 새로운 기회를 제공하는 동시에, 기존과는 다른 보안 문제를 야기한다. LLM 보안은 기존의 '경계 보안' 개념을 넘어, '입력-모델-출력'으로 이어지는 전체 흐름에서 취약점을 진단하고 방어하는 동적인 영역이 되었다.

따라서 LLM을 활용하는 서비스는 개발 단계부터 **프롬프트 인젝션 방어**, **민감 정보 필터링** 등을 최우선으로 고려해야 한다. LLM의 **환각(Hallucination)** 현상과 같은 신뢰성 문제도 중요한 보안 이슈로 인식하고, RLHF(사람의 피드백을 통한 강화학습)와 같은 기술을 통해 지속적으로 모델의 안정성을 개선해야 할 것이다.

또한, LLM 보안의 성공은 기술적인 방어책뿐만 아니라 사용자와 개발자 모두의 **보안 교육**에 달려 있다고 볼 수 있을 것이다. 특히 일반 사용자는 LLM에 **민감한 개인정보를 입력하지 않도록** 주의하고, LLM의 답변이 **항상 진실이 아닐 수 있다**는 점을 인지하여 비판적인 시각을 가져야 한다. 개발자 및 운영자는 안전한 프롬프트 작성법, 보안 설정 체크리스트, 책임 있는 LLM 활용에 대한 정기적인

교육을 통해 모든 구성원이 보안 의식을 갖추도록 해야한다.

결론적으로, LLM 보안은 단순히 기술적인 방어책을 마련하는 것을 넘어, **모델의 윤리적 사용과 사회적 책임**을 함께 고려해야 하는 종합적인 과제라고 생각한다.

참고 자료

1. OWASP. "OWASP LLM Top 10". OWASP Foundation,
<https://owasp.org/www-project-top-10-for-large-language-model-applications/>.
2. SK Shielders. "SK 쉴더스 LLM Application 취약점 진단 가이드"
https://www.skshieldus.com/download/files/download.do?o_fname=LLM%20Application%20%EC%B7%A8%EC%95%BD%EC%A0%90%20%EC%A7%84%EB%8B%A8%20%EA%B0%80%EC%9D%B4%EB%93%9C.pdf&r_fname=20241129161501834.pdf